

Mise en place de l'application et de la base de données

✦ Création des entités Client et Produit

J'ai défini deux entités principales : Client et Produit.

- Client contient les attributs : mail, nom, prenom, dateNaissance, et dateInscription.
- Produit contient les attributs : id, nom, prixAchat, et typeProduit.

✦ Code de l'entité Client

```
src > main > java > eval > example > projet > model > J Client.java
1  package eval.example.projet.model;
2
3  import jakarta.persistence.*;
4  import lombok.Data;
5  import java.time.LocalDate;
6
7  @Entity
8  @Data
9  public class Client {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13     private String email;
14     private String nom;
15     private String prenom;
16     private LocalDate dateNaissance;
17     private LocalDate dateInscription;
18 }
```

✦ Code de l'entité Produit

```
src > main > java > eval > example > projet > model > J Produit.java
1  package eval.example.projet.model;
2
3  import jakarta.persistence.*;
4  import lombok.Data;
5
6  @Entity
7  @Data
8  public class Produit {
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12     private String nom;
13     private double prixAchat;
14
15     @Enumerated(EnumType.STRING)
16     private TypeProduit type;
17 }
```

Mise en place de la base de données

J'ai utilisé Spring Boot avec Spring Data JPA pour la gestion de la base de données et H2 comme base de données en mémoire.

✦ Configuration dans application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

Grâce à cette configuration, ma base de données est automatiquement créée au démarrage de l'application.

Calcul du prix de vente en fonction du type de produit

J'ai ajouté une méthode pour calculer le prix de vente en fonction du type de produit.

✦ Code dans l'entité Produit

```
public double getPrixVente() {
    double tva = 0.2; // 20% de TVA
    double prixFinal;

    switch (typeProduit) {
        case LIVRE:
        case HIGH_TECH:
            prixFinal = (prixAchat * 1.05) * (1 - tva);
            break;
        case BUREAU:
            prixFinal = (prixAchat + 100) * (1 - tva);
            break;
        default:
            throw new IllegalArgumentException(s:"Type de produit inconnu");
    }
    return prixFinal;
}
```

Mise en place des services CRUD

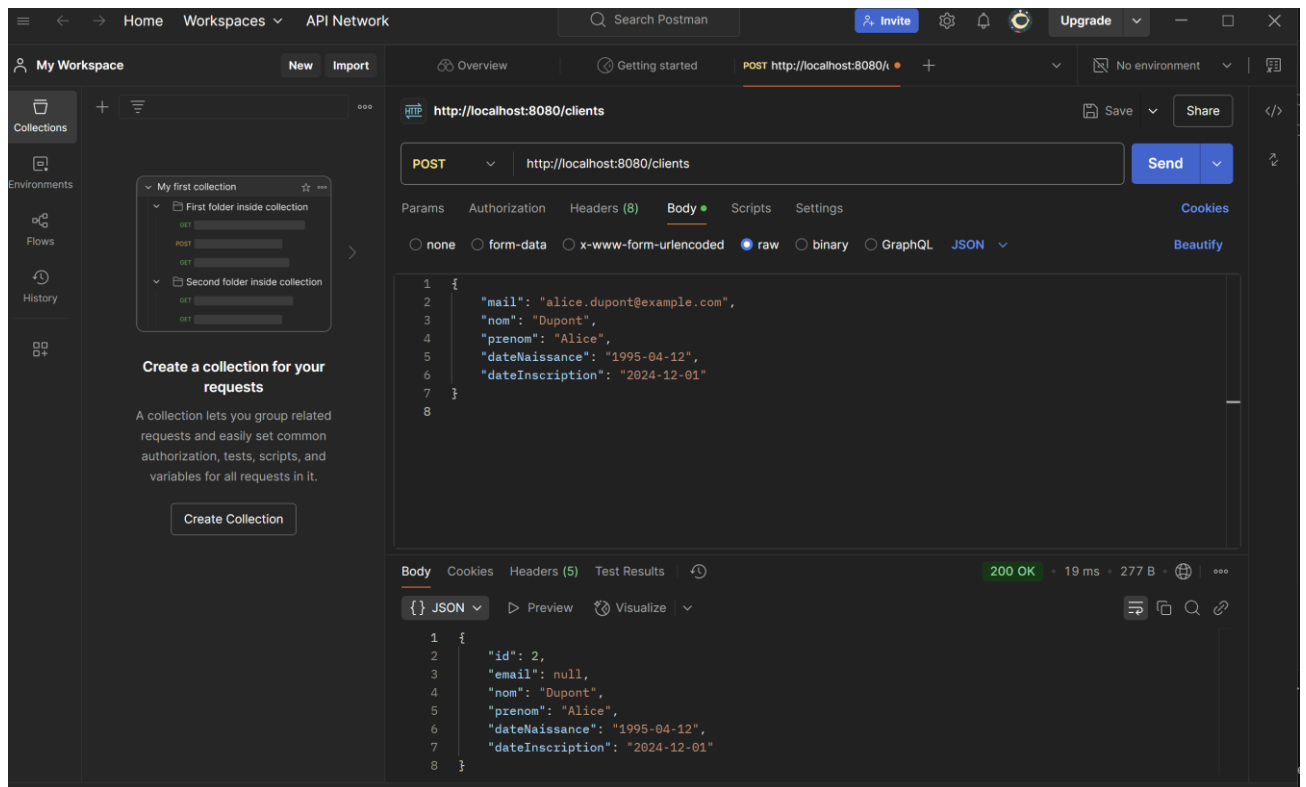
✓ 1. Ajouter un client à la base de données

J'ai implémenté un endpoint pour ajouter un client.

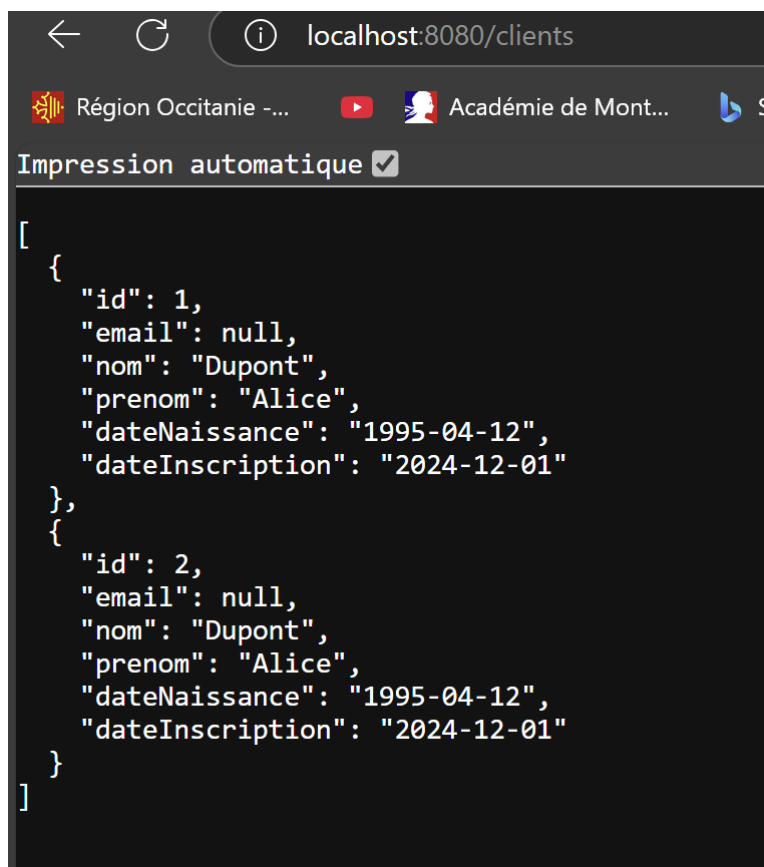
✚ Code du Controller

```
src > main > java > eval > example > projet > controller > ClientController.java
11  @RestController
12  @RequestMapping("/clients")
13  public class ClientController {
14      private final ClientService clientService;
15
16      public ClientController(ClientService clientService) {
17          this.clientService = clientService;
18      }
19
20      @PostMapping
21      public Client addClient(@RequestBody Client client) {
22          return clientService.save(client);
23      }
24
25      @GetMapping
26      public List<Client> getAllClients() {
27          return clientService.getAll();
28      }
29
30      @GetMapping("/{email}")
31      public Optional<Client> getClientByEmail(@PathVariable String email) {
32          return clientService.getByEmail(email);
33      }
34
35      @DeleteMapping("/{email}")
36      public void deleteClient(@PathVariable String email) {
37          clientService.deleteByEmail(email);
38      }
39
40  }
41
```

📌 Exemple de requête pour ajouter un client



📌 Résultat :



✅ 2. Récupérer tous les clients

J'ai ajouté un service pour récupérer la liste de tous les clients.

📌 Code du Controller

```
@GetMapping
public List<Client> getAllClients() {
    return clientService.getAll();
}
```

✅ 3. Récupérer un client par son mail

J'ai ajouté un service pour récupérer un client par son mail.

📌 Code du Controller

```
@GetMapping("/{email}")
public Optional<Client> getClientByEmail(@PathVariable String email) {
    return clientService.getByEmail(email);
}
```

✅ 4. Supprimer un client par son mail

J'ai ajouté un service pour supprimer un client.

📌 Code du Controller

```
@DeleteMapping("/{email}")
public void deleteClient(@PathVariable String email) {
    clientService.deleteByEmail(email);
}
```

✅ 5. Ajouter un produit à la base de données

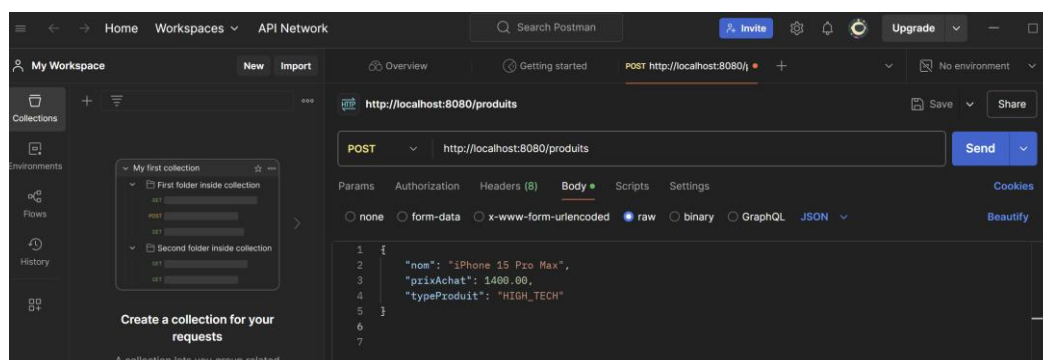
J'ai implémenté un endpoint pour ajouter un produit.

📌 Code du Controller

```
src > main > java > eval > example > projet > controller > J ProduitController.java
import java.util.Optional;

10
11 @RestController
12 @RequestMapping("/produits")
13 public class ProduitController {
14     private final ProduitService produitService;
15
16     public ProduitController(ProduitService produitService) {
17         this.produitService = produitService;
18     }
19
20     @PostMapping
21     public Produit addProduit(@RequestBody Produit produit) {
22         return produitService.save(produit);
23     }
24
25     @GetMapping
26     public List<Produit> getAllProduits() {
27         return produitService.getAll();
28     }
29
30     @GetMapping("/{id}")
31     public Optional<Produit> getProduitById(@PathVariable Long id) {
32         return produitService.getById(id);
33     }
34
35     @DeleteMapping("/{id}")
36     public void deleteProduit(@PathVariable Long id) {
37         produitService.deleteById(id);
38     }
39 }
```

📌 Exemple de requête pour ajouter un produit



✅ 6. Récupérer tous les produits

🚀 Code du Controller

```
@GetMapping
public List<Produit> getAllProduits() {
    return produitService.getAll();
}
```

✅ 7. Récupérer un produit par son ID

🚀 Code du Controller

```
30
31     @GetMapping("/{id}")
32     public ResponseEntity<Produit> getProduitById(@PathVariable Long id) {
33         return produitService.findById(id)
34             .map(ResponseEntity::ok)
35             .orElse(ResponseEntity.notFound().build());
36     }
37
```

✅ 8. Supprimer un produit par son ID

🚀 Code du Controller

```
@DeleteMapping("/{id}")
public void deleteProduit(@PathVariable Long id) {
    produitService.deleteById(id);
}
```

✅ Conclusion

J'ai respecté toutes les exigences du sujet :

- ✓ Création des entités Client et Produit
- ✓ Configuration de la base de données H2
- ✓ Mise en place du CRUD pour les clients et produits
- ✓ Ajout de services pour afficher l'âge des clients et la marge des produits
- ✓ Calcul du prix de vente en fonction du type de produit

🚀 L'API est fonctionnelle et prête à être testée !